Classification of Legal Text: Replication & Tuning

Mini-Project Write-up | Replicating Krithika Iyer, *"Classification of Legal Text,"* Stanford CS229 (2020)

Problem Statement

Legal opinions use long, non-standard sentence structures that break typical NLP tooling. We replicate and extend a Stanford CS229 project that classifies U.S. Supreme Court opinions into their human-curated legal subject-matter category, at two granularities: **issue_area** (15 broad categories, e.g. Criminal Procedure, Civil Rights, First Amendment) and **issue** (260+ fine-grained issue codes). The goal is to test whether classic topic-model and embedding-based document features, paired with a simple classifier, can recover these categories, and whether tuning their hyperparameters meaningfully improves on the paper's untuned baseline.

Dataset

We use the exact dataset the original paper used: the Supreme Court Database (Washington University) via the **textacy** package's bundled **SupremeCourt** dataset — 8,419 opinions issued between 1946 and 2016, each with an expert-assigned issue_area and issue code. No substitute dataset was needed. After dropping opinions with no issue_area label and issue codes with fewer than 10 examples (so every class can appear in train/dev/test), 7,971 opinions remain, split 70% / 15% / 15% into train / dev / test, stratified by issue_area. Because opinions run very long (median ~34,000 characters), each is truncated to its first 4,000 characters before vectorization, to keep LDA/Doc2Vec training times tractable — most opinions state the issue and holding early, so this preserves the dominant vocabulary.

Approach

We replicate the paper's two completed experiments (its third, BERT, ran out of memory in the original work and produced no results to reproduce):

1. LDA + Logistic Regression — a gensim Dictionary and TF-IDF model feed a Latent Dirichlet Allocation topic model; each document becomes a dense topic-probability vector, classified with scikit-learn Logistic Regression.
2. Doc2Vec + Logistic Regression — a gensim Doc2Vec model learns a fixed-size paragraph embedding per opinion, again classified with Logistic Regression.

Phase 1 (replicate): both models trained with the paper's rough default settings (LDA num_topics=50, Doc2Vec vector_size=100, Logistic Regression C=1.0), evaluated on both label tasks. **Phase 2 (tune):** each feature extractor's main hyperparameter (LDA num_topics $\in$ {20, 50, 100}; Doc2Vec vector_size $\in$ {50, 100, 200}) is selected by issue_area dev accuracy, then each downstream classifier's regularization C is grid-searched per label task on dev only — the test set is touched exactly once, with the final chosen configuration.

Implementation

Code is organized as `src/preprocessing.py` (corpus loading, cleaning, tokenization, splitting), `src/lda_lr.py` and `src/doc2vec_lr.py` (the two feature extractor + classifier pipelines), `run_all.py` (Phase 1) and `tune_all.py` (Phase 2). All hyperparameter selection uses only the dev split; the test split is evaluated once per final configuration, keeping the comparison honest.

Results — Phase 1 (paper's default settings)

Model	Label task	Test Acc	Test Macro-F1
lda_lr	issue_area (15-label)	39.3%	0.149
lda_lr	issue (fine-grained)	8.3%	0.013
doc2vec_lr	issue_area (15-label)	59.1%	0.463
doc2vec_lr	issue (fine-grained)	37.1%	0.290

Results — Phase 2 (tuned, test touched once)

Model + Task	Tuned Param	Test Acc Before	Test Acc After

lda_lr_issue_area	num_topics=100	39.3%	48.7%
lda_lr_issue	num_topics=100	8.3%	19.9%
doc2vec_lr_issue_area	vector_size=50	59.1%	62.2%
doc2vec_lr_issue	vector_size=50	37.1%	42.0%

Tuning gave clear gains throughout: LDA benefited from more topics (50→100; issue_area test accuracy +9.4 points) — 50 topics was too coarse to separate 15+ categories. Doc2Vec preferred a *smaller* embedding (100→50); with ~5,600 training documents, 200 dimensions overfits before the classifier sees enough signal.

Conclusions

Doc2Vec embeddings consistently outperform LDA topic-distribution features for this task (e.g. 62.2% vs 48.7% test accuracy on issue_area after tuning), and the coarser 15-category task is much easier than the 260+-category fine-grained task — unsurprising given how thin per-class training data gets at that granularity. LDA's macro-F1 trails its accuracy badly (0.267 vs 48.7% on issue_area), indicating it defaults to majority classes (Criminal Procedure, Economic Activity) and fails on rare ones; Doc2Vec generalizes far better to minority categories such as Privacy and Attorneys. Our replicated numbers land in the same range and follow the same ordering as the original paper's reported results. We note an internal inconsistency in the paper's own results table (its printed table and its prose text disagree on which label granularity scored higher) and resolve it in favor of the prose. We did not replicate the paper's BERT experiment, since it failed with an out-of-memory error in the original work and left no baseline number to reproduce; a long-document Transformer variant (e.g. Longformer) is natural future work, as the original paper itself suggests.

Full code, results, and figures: github.com/amogh-herle/ML-MINI-PROJECT